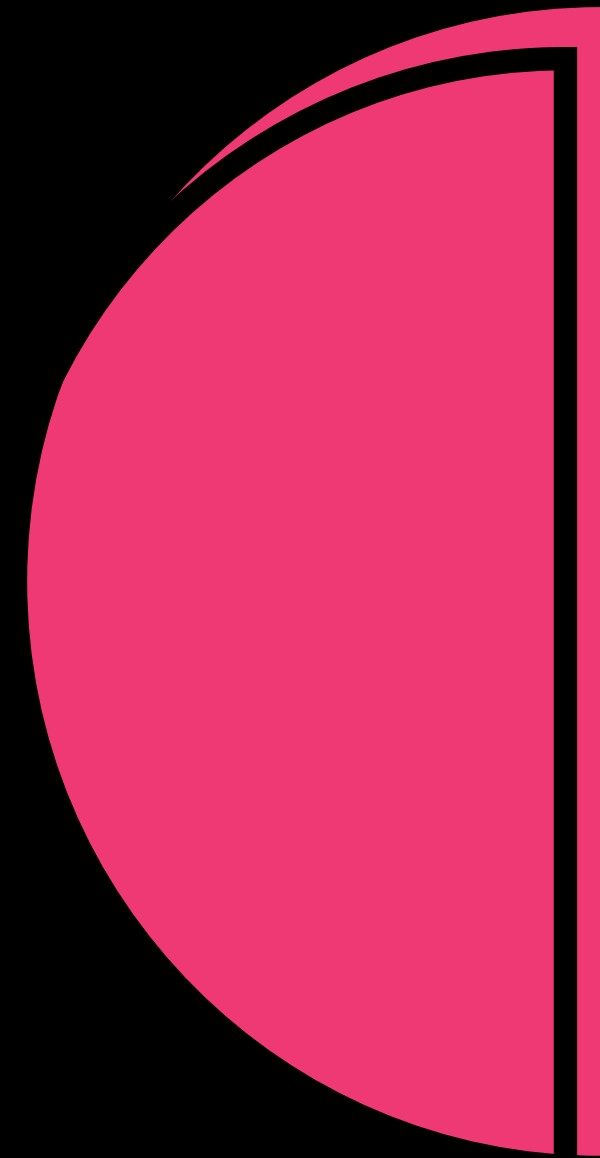




Start

PROJECT 2: DIJKSTRA'S ALGORITHM

(a) Adjacency Matrix + Array
Implementation of Dijkstra with
array as priority queue

(b) Adjacency List + Min-Heap
Implementation of Dijkstra
with min heap as priority queue

(c) Comparison of two methods
Analyse time complexity and
decide which algorithm is better

(a) Adjacency Matrix + Array for Priority Queue

- Graph representation: **Adjacency Matrix**
 - Graph stored as a 2D matrix
 - Easy to check edge existence in $O(1)$ time.
 - Requires $O(|V|^2)$ space since we use space even for non existing edges, better for **dense** graphs
- Priority Queue implementation: **Array**
 - Uses an array to track vertex distances
 - Finding the minimum distance vertex takes $O(|V|)$ per step.
 - All vertices are scanned for every iteration
- Time Complexity
 - **$O(|V|^2)$**
 - Independent of $|E|$ since all pairs are checked

(a) Adjacency Matrix + Array for Priority Queue

```
def dijkstra_matrix_array(adjMatrix, sourceNode):
    d = [] # array of estimates for lengths of shortest paths from source node
    pi = [] # array of predecessors for each vertex
    S = [] # set of vertices whose shortest path from source already determined
    priorityQueue = [] # Priority Queue Q
    infinity = float('inf')
    graphLength = len(adjMatrix)

    for i in range(graphLength):
        d.append(infinity)
        pi.append(-1) # set predecessor to NULL, pi[v] = null
        S.append(0) # set solution set S to empty at first, where all v not in S
        priorityQueue.append(i)

    d[sourceNode] = 0 # set estimated distance to itself (src to src) to be 0

    while len(priorityQueue) != 0:
        cheapest = 0
        for i in range(len(priorityQueue)):
            if (d[priorityQueue[i]] < d[priorityQueue[cheapest]]):
                cheapest = i

        u = priorityQueue.pop(cheapest)
        S[u] = 1 # add u to S

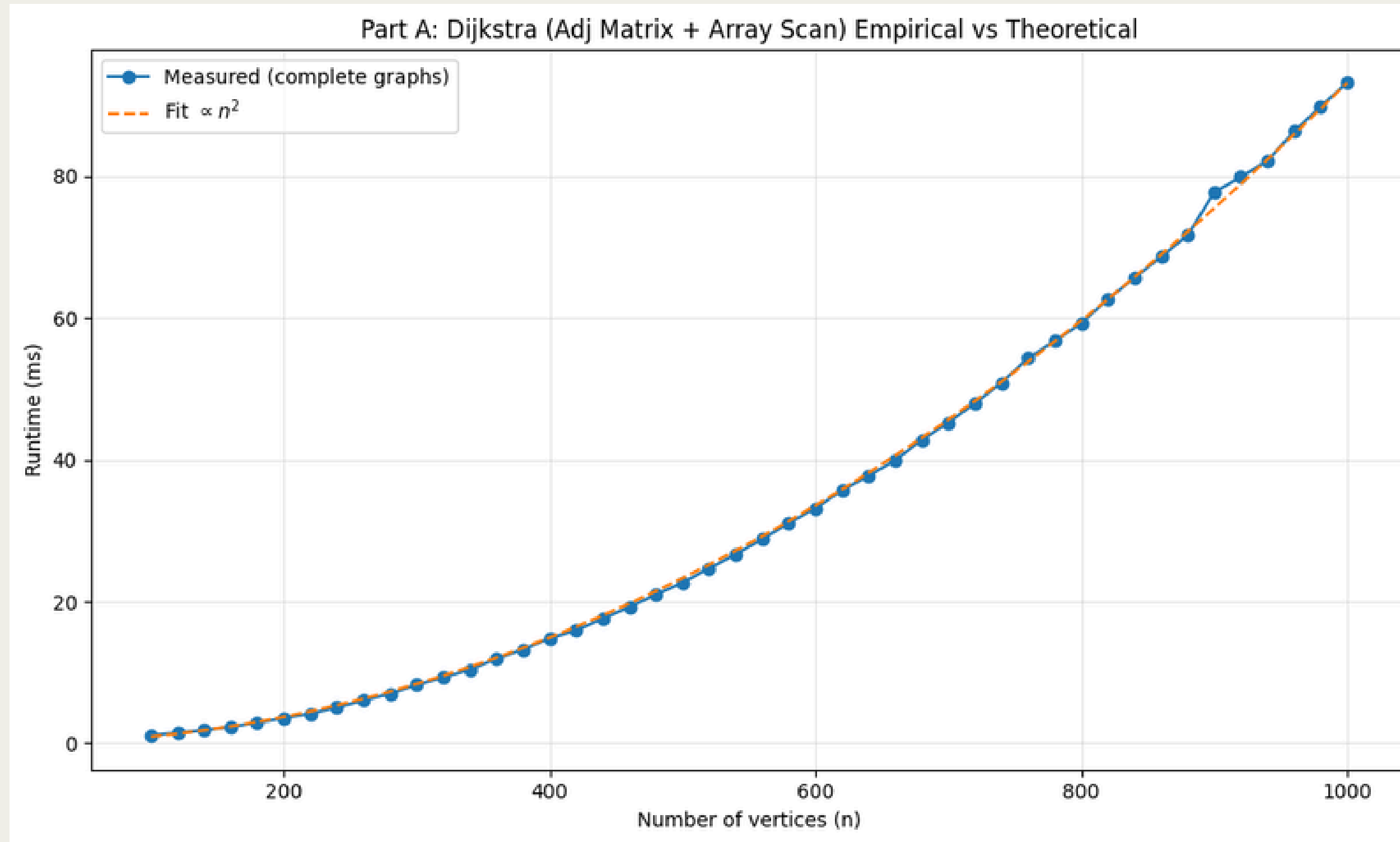
        for i in range(graphLength): # for each vertex v adjacent to u
            vertex = i
            weight = adjMatrix[u][i]
            if (weight > 0):
                # if vertex v not in solution set & estimated distance
                # of vertex v bigger than src node + weighted edge[u, v]
                if ((S[vertex] != 1) and (d[vertex] > (d[u] + weight))):
                    d[vertex] = d[u] + weight # update the new shortest value
                    pi[vertex] = u # set predecessor of vertex v as u

    return d, pi
```

—————→ $O(V^2)$ total as while loop runs V times

—————→ $O(V)$ in the worst case

—————→ $O(V)$ per loop



Actual runtime follows **closely** to theoretical runtime as seen by the closeness between the blue and orange dotted lines

(b) Adjacency List + Min-Heap for Priority Queue

- Graph representation: **Adjacency List**
 - Efficient for sparse graphs since we only store existing edges.
 - Each vertex maintains list of its neighbours and edge weights
 - Requires $O(|V| + |E|)$ space.
- Priority Queue implementation: **Min-Heap**
 - Uses a min-heap (priority queue) to efficiently extract the vertex with the smallest tentative distance.
 - Each insertion or decrease-key operation on the heap takes $O(\log |V|)$ time.
 - Extracting the minimum vertex also takes $O(\log |V|)$.
- Time Complexity:
 - Each vertex is inserted into the heap once $\rightarrow O(|V| \log |V|)$
 - Each edge relaxation may cause a decrease-key operation $\rightarrow O(|E| \log |V|)$
 - Total time complexity = $O((|V| + |E|) \log |V|)$

(b) Adjacency List + Min-Heap for Priority Queue

```
# -----  
# Dijkstra's algorithm using adjacency list + min-heap  
# -----  
def dijkstra(graph, source):  
    distances = {vertex: float('inf') for vertex in graph}  
    distances[source] = 0  
    pq = [(0, source)] # (distance, vertex)  
  
    while pq:  
        current_distance, current_vertex = heapq.heappop(pq) ->  
        if current_distance > distances[current_vertex]:  
            continue  
  
        for neighbor, weight in graph[current_vertex]:  
            distance = current_distance + weight  
            if distance < distances[neighbor]:  
                distances[neighbor] = distance  
                heapq.heappush(pq, (distance, neighbor)) ->  
  
    return distances
```

Time Complexity: $O((|V| + |E|) \log |V|)$

Breakdown:

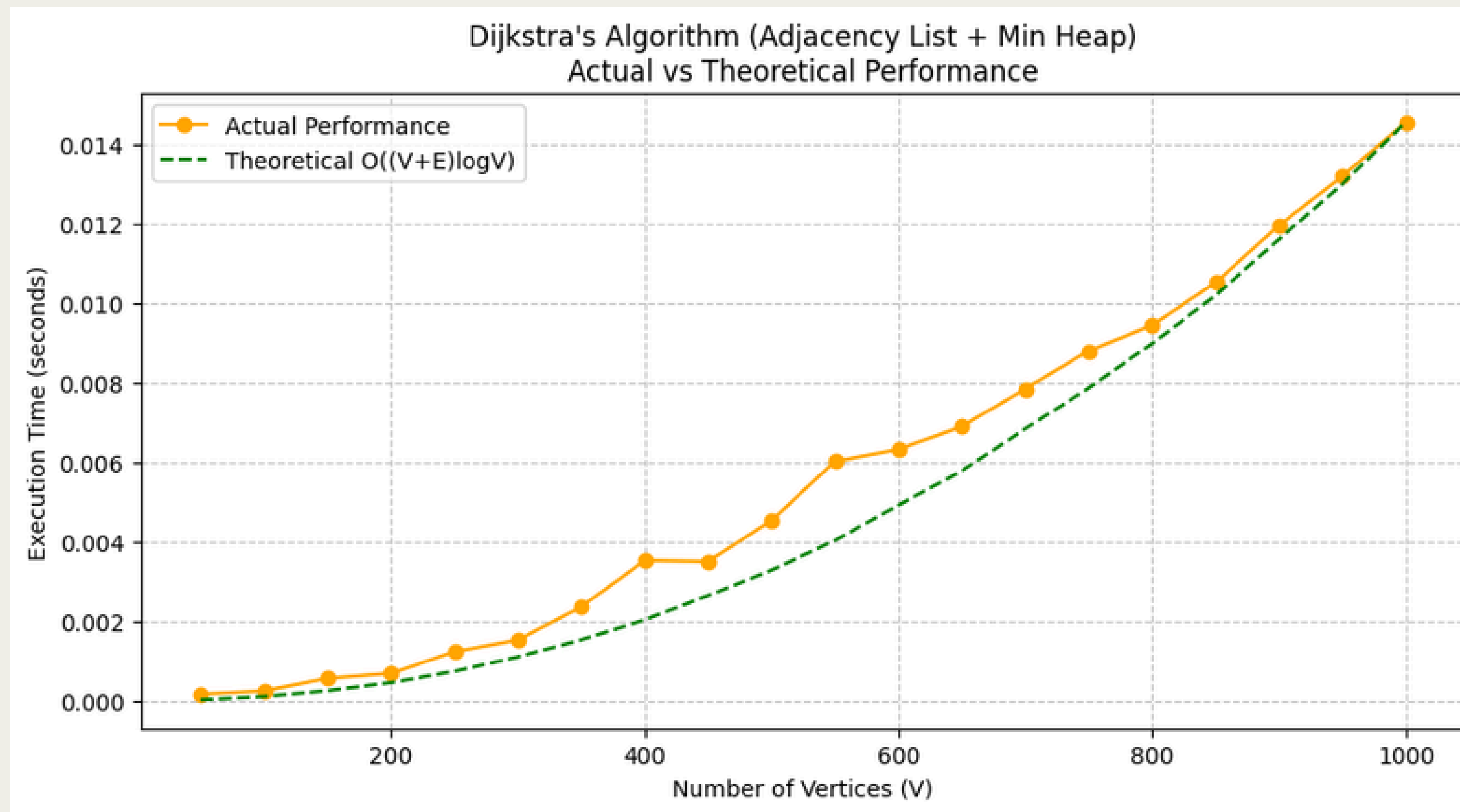
$|V|$ heap extractions $\times O(\log |V|) = O(|V| \log |V|)$

$|E|$ heap insertions $\times O(\log |V|) = O(|E| \log |V|)$

Total: $O((|V| + |E|) \log |V|)$

happens $|V|$ times $\rightarrow O(|V| \log |V|)$

happens $|E|$ times $\rightarrow O(|E| \log |V|)$



Observations:

- Sub-quadratic growth confirms $O((V+E) \log V)$
- Near-linear appearance due to slow $\log V$ growth

(c) Comparison

- **Theoretical Time Complexity:**

- Adjacency Matrix + Array Priority Queue: $O(V^2)$
- Adjacency List + Min-Heap Priority Queue: $O((V + E) \log V)$
- Density $\approx E/V^2$

- **SPARSE Graph:**

- **Adjacency List** only processes existing edges
- Density small $\rightarrow E \ll V^2 \rightarrow E = cV$ where c is small
- Time Complexity for Matrix = $O(V^2)$
- Time Complexity for List = $O((V+E) \log V) = O((c+1)V \log V) = O(V \log V)$

- **DENSE Graph:**

- **Adjacency Matrix** uses simple array access and avoids heap overhead
- Density large $\rightarrow E \approx V^2$ (get close to V^2)
- Time Complexity for Matrix = $O(V^2)$
- Time Complexity for List = $O((V + E) \log V) = O((V + V^2) \log V) = O((V^2) \log V)$

(c) Runtime analysis - Sparse, Average, Dense graphs

1. Sparse Graphs

- Sparse graphs have very few edges compared to vertices ($E \ll V^2$).
 - The **Adjacency Matrix** checks all possible edges, leading to a fixed cost of $O(V^2)$.
 - The **Adjacency List (Min-Heap)** only processes existing edges with $O((V + E) \log V)$ operations.
-

2. Average Graphs

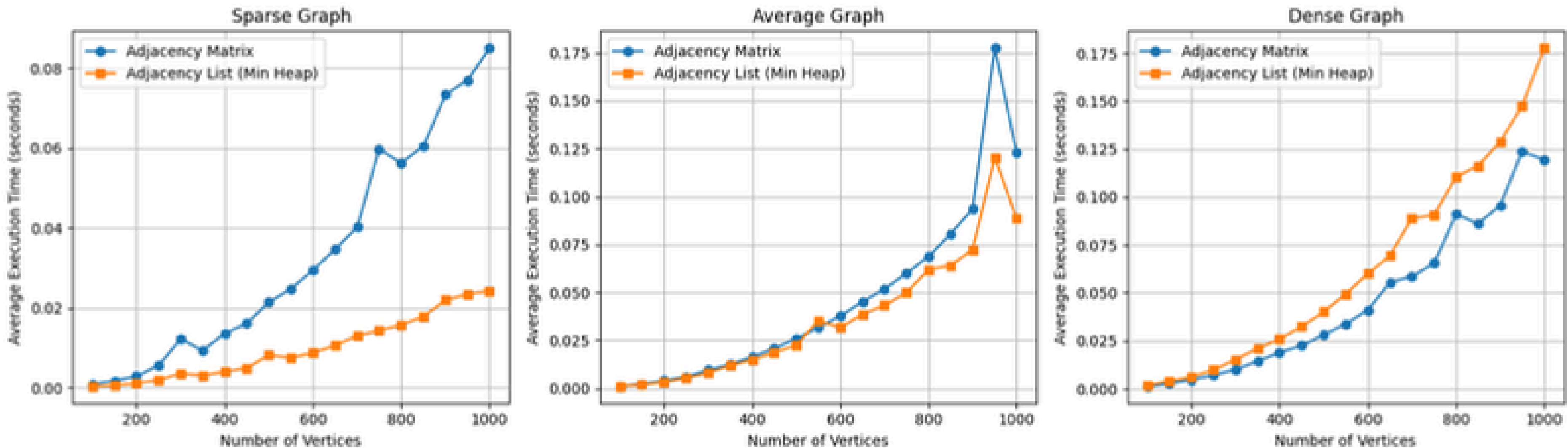
- As graphs become denser, both methods take longer to finish.
 - The **list-based** version performs more heap operations as E increases, while the **matrix-based** version remains $O(V^2)$.
 - Their runtimes **get closer**, but the list version stays slightly faster due to smaller constant factors.
-

3. Dense Graphs

- Dense graphs have $E \approx V^2$.
- Both need to explore almost every edge, but the **Adjacency Matrix** is quicker because it uses simple array access and avoids heap overhead.
- The **Min-Heap** performs many heap operations ($\log V$ each), making it slower.

(c) Runtime comparisons of two Implementations

Dijkstra's Algorithm: Adjacency Matrix vs Adjacency List



CONCLUSION

- For **sparse** graphs, using an **minimizing heap** as priority queue would outperform that of an array.
- For **dense** graphs, using an **array-based** priority queue is better than that of a min-heap.
- Still, on **average**, using a **minimizing heap** would work more efficiently than an array.



THANK YOU!

